# IT Help Desk Labs 6 - 8

Network Troubleshooting, Remote Access, Ticket Handling and Imaging

| Lab | Title | Environment |
|---|---|---|
| 6 | Network Isolation & Remote Access: USB Tethering, NAT Boundaries, AnyDesk Workaround | Linux Mint laptop, Windows 10 PC, smartphone (home lab) |
| 7 | DNS / Network Connectivity Troubleshooting | ServiceDesk Simulator, Windows 10 |
| 8 | Dead PC Ticket, Replacement and Network Image Deployment | ServiceDesk Simulator |

# Lab 6: Network Isolation & Remote Access

## USB Tethering, NAT Boundaries, and the AnyDesk Workaround

## Summary

I wanted to share a mouse and keyboard between a Linux Mint laptop and a Windows 10 PC that has no Wi-Fi. The Windows PC only had internet through an old smartphone using **USB tethering**. KDE Connect and Barrier both failed because the two machines ended up on **different subnets behind a NAT boundary**. I documented why it failed, then solved the real goal (controlling one PC from the other) with **AnyDesk**, which does not need both machines on the same network.

**Skills shown:** network troubleshooting, subnets/NAT/DHCP concepts, ipconfig / ifconfig diagnostics, Windows network profiles and firewall, Linux command line and config files, package installation, remote desktop tooling, documentation.

## 1. Environment

| Role | Device | Connection |
|---|---|---|
| Primary host | Lenovo Yoga 900 running Linux Mint | Home Wi-Fi |
| Secondary host | Windows 10 PC (no native Wi-Fi) | USB tethering through a smartphone (adapter shown as `Ethernet 3`) |
| Gateway for Windows PC | Old smartphone | Home Wi-Fi upstream, USB tether downstream |

**Goal:** share mouse/keyboard and move between the two PCs without building VPNs or static routes.

## 2. Topology and the "NAT Wall"

```
[ Home Wi-Fi Router ]
        |
        |-----> [ Linux Mint Laptop ]            (primary subnet)
        |
        '-----> [ Smartphone ] --(USB tether / NAT)--> [ Windows PC ]
                                                       (192.168.42.x)
```

- The phone runs its own DHCP server and NAT for the tethered PC, so the Windows PC received a private address in the `192.168.42.x` range.
- The Linux laptop sits on the home router's subnet. The two machines are not in the same broadcast domain.

**Why discovery failed**

- **Layer 2:** KDE Connect finds devices with UDP broadcast. Broadcasts do not cross the phone's NAT.
- **Layer 3:** Entering the Windows PC's IP manually (customDevices) does not help, because the home router has no route to the phone's internal `192.168.42.x` network, so traffic from the laptop cannot reach the PC.

## 3. What I tried

| # | Attempt | Result |
|---|---|---|
| 1 | Installed KDE Connect on both PCs and tried auto-discovery | Windows showed "No devices found" |
| 2 | Ran ipconfig on Windows to find the tether adapter's IPv4 address, subnet mask and gateway | Confirmed the PC was on a separate 192.168.42.x network |
| 3 | Ran ifconfig on Linux to compare interfaces | Confirmed different subnets |
| 4 | Set the Windows Ethernet 3 profile to **Private** so the PC is discoverable | Profile changed correctly, discovery still failed |
| 5 | Created a KDE Connect customDevices config on Linux pointing at the Windows IP | Path typo found (see below); no connection either way |
| 6 | Installed **Barrier** on Linux (Server) for input sharing with Windows (Client) | Package installed, but TCP still could not cross the NAT boundary reliably |

## Troubleshooting detail: the config path typo

My first command printed `mkdir: command not found`, so it had been mistyped. When I opened the file in nano, it reported that the directory did not exist. The path read `/home/lavi/,config/kdeconnect` with a **comma instead of a dot**. The correct path is `~/.config/kdeconnect/config`. Even with the file correct, packets cannot reach `192.168.42.x` from the home subnet, so the connection still failed.

```
# Linux
ifconfig
mkdir -p ~/.config/kdeconnect
nano ~/.config/kdeconnect/config
killall kdeconnectd
sudo apt install barrier

:: Windows
ipconfig
```

## 4. Decision

Fixing this properly meant static routes, a VPN, or a tunnel. That is a lot of overhead for sharing a mouse, so I stopped and documented the boundary instead of forcing it.

## 5. Resolution: AnyDesk

I installed **AnyDesk** on both machines (Windows 10 through the browser download, Linux Mint with AnyDesk 8.1) and connected the two PCs. This works because AnyDesk connects through its own ID-based service over the internet instead of relying on local network discovery. Both PCs already had internet access (one via Wi-Fi, one via the tethered phone), so the subnet split no longer mattered.

**Trade-offs I noted**

- Works across subnets and NAT with no router or routing changes.

- Needs internet on both sides; it is not a purely local connection.
- It is remote desktop, not seamless edge-of-screen mouse sharing like Barrier would be on one LAN.
- Free licence is for non-professional use only.

## 6. Key takeaways

1. Peer-to-peer discovery tools depend on a shared broadcast domain; a NAT boundary breaks that.
2. Manually entering an IP only helps if there is a routed path to it.
3. Checking IP, mask and gateway on both machines (ipconfig, ifconfig) quickly shows when two devices are on different networks.
4. Typos in paths matter: one wrong character in a config path hid the real problem for a while.
5. Sometimes the best fix is to change the tool (AnyDesk) rather than rebuild the network.

## 7. If I extended this lab

- Put both PCs on the same router (Wi-Fi USB adapter for the Windows PC) and retest KDE Connect/Barrier.
- Test Windows built-in Remote Desktop over a routed path.
- Compare AnyDesk with another remote tool for latency and security settings.

# Lab 7: DNS / Network Connectivity Troubleshooting

Environment: ServiceDesk Simulator on Windows 10

## Summary

Using ServiceDesk Simulator on Windows 10, I fixed a connection problem where the browser could not load websites and showed a "You're not connected" error. I used Command Prompt to check the network adapters, flush the DNS cache, and release and renew the IP address. After that, connectivity was restored.

## Problem and fix

| Symptom | Action | Result |
|---|---|---|
| Browser could not load websites; "You're not connected" error | Checked network adapters in Command Prompt, then ran the commands below | Connectivity restored |

```
ipconfig /flushdns
ipconfig /release
ipconfig /renew
```

## Steps taken

1. Confirmed the symptom: websites would not load and the browser reported no connection.
2. Opened Command Prompt and checked the network adapters.
3. Flushed the DNS cache with `ipconfig /flushdns`.
4. Released and renewed the IP address with `ipconfig /release` and `ipconfig /renew`.
5. Retested and confirmed the browser could load websites again.

## Skills shown

Network troubleshooting, DNS, IP configuration, command line.

# Lab 8: Dead PC Ticket, Replacement and Network Image Deployment

Environment: ServiceDesk Simulator

## Summary

In ServiceDesk Simulator I handled a Critical ticket (**INC0012871**) for a Customer Support PC that would not power on. I assigned the ticket to myself, chatted with the user to gather information, and ruled out the power cable. When no spare PC was available in asset management, I deployed a new image over the network (PXE boot and task sequence) and followed the SOP for naming the machine. I then shipped the PC to the user, and she confirmed it worked.

## Ticket details

| Field | Detail |
|---|---|
| Ticket | INC0012871 |
| Priority | Critical |
| Issue | Customer Support PC would not power on |
| Outcome | Replacement PC deployed and shipped; user confirmed it worked |

## Steps taken

1. Assigned the ticket to myself.
2. Chatted with the user to gather information about the problem.
3. Ruled out the power cable as the cause.
4. Checked asset management for a spare PC; none was available.
5. Deployed a new image over the network using PXE boot and a task sequence.
6. Named the machine following the SOP.
7. Shipped the PC to the user.
8. Received confirmation from the user that it worked.

## Skills shown

Ticket handling, user communication, hardware troubleshooting, asset management, PXE network imaging, following SOPs.